S.No: 5	Exp. Name: ***Implement Sum of Subset Problem Using Backtracking***	Date:2026-07-10

ID: 25070521290

Aim:

You are given a set of integers and a target sum. Write a program that finds all subsets of the given set whose sum is equal to the target sum.

Input Format:

- The first line contains an integer, n the number of elements in the set.
- The second line contains n space-separated integers representing the elements of the set.
- The third line contains an integer, sum the target sum.

Output Format:

- Print all subsets whose sum is equal to the given target sum. Each subset should be printed in a new line, and the elements within the subset should be separated by a space.
- If no such subset exists, print "-1"
- The subsets should be printed in reverse order of their discovery (i.e., the last found subset should be printed first).

Example 1:

Input:

```
3
1 2 1
3
```

Output:

```
2 1
1 2
```

Example 2:

Input:

```
6
3 34 4 12 5 2
30
```

Output:

```
-1
```

Constraints:

- $1 \leq n \leq 20$
- $1 \leq element \leq 100$
- $1 \leq sum \leq 1000$

Source Code:

CTC64796.c

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
```

```
bool foundAny = false;

// Dynamic array structure to store found subsets
int **allSubsets = NULL;
int *subsetSizes = NULL;
int subsetCount = 0;
int maxSubsets = 2000;

// Backtracking function moving from index 0 to n-1 (Include before Exclude)
void findSubsets(int arr[], int n, int index, int target, int currentSubset[], int su
bsetSize) {
    // Base Case: Target sum reached
    if (target == 0) {
        foundAny = true;

        // Save the found subset to print in reverse later
        allSubsets[subsetCount] = (int *)malloc(subsetSize * sizeof(int));
        subsetSizes[subsetCount] = subsetSize;
        for (int i = 0; i < subsetSize; i++) {
            allSubsets[subsetCount][i] = currentSubset[i];
        }
        subsetCount++;
        return;
    }

    // Base Case: Out of bounds or target exceeded
    if (index == n || target < 0) {
        return;
    }

    // 1. INCLUDE the current element first
    currentSubset[subsetSize] = arr[index];
    findSubsets(arr, n, index + 1, target - arr[index], currentSubset, subsetSize +
1);

    // 2. EXCLUDE the current element second
    findSubsets(arr, n, index + 1, target, currentSubset, subsetSize);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) {
            free(arr);
            return 0;
        }
    }

    int targetSum;
    if (scanf("%d", &targetSum) != 1) {
        free(arr);
        return 0;
    }
```

```
    // Pre-allocate tracking buffers for collected matrices
    allSubsets = (int **)malloc(maxSubsets * sizeof(int *));
    subsetSizes = (int *)malloc(maxSubsets * sizeof(int));
    int *currentSubset = (int *)malloc(n * sizeof(int));

    // Run forward discovery backtracking
    findSubsets(arr, n, 0, targetSum, currentSubset, 0);

    // If matches exist, print them starting from the LAST found subset down to the f
irst
    if (foundAny) {
        for (int i = subsetCount - 1; i >= 0; i--) {
            for (int j = 0; j < subsetSizes[i]; j++) {
                // FIXED: Prints a space after EVERY single integer to match "2·1·" f
ormat
                printf("%d ", allSubsets[i][j]);
            }
            printf("\n");
        }
    } else {
        printf("-1\n");
    }

    // Free all allocated memory blocks cleanly
    for (int i = 0; i < subsetCount; i++) {
        free(allSubsets[i]);
    }
    free(allSubsets);
    free(subsetSizes);
    free(currentSubset);
    free(arr);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
1 2 1
3
2 1
1 2

Test Case - 2
User Output
6
3 34 4 12 5 2
30
-1

ID: 25070521290